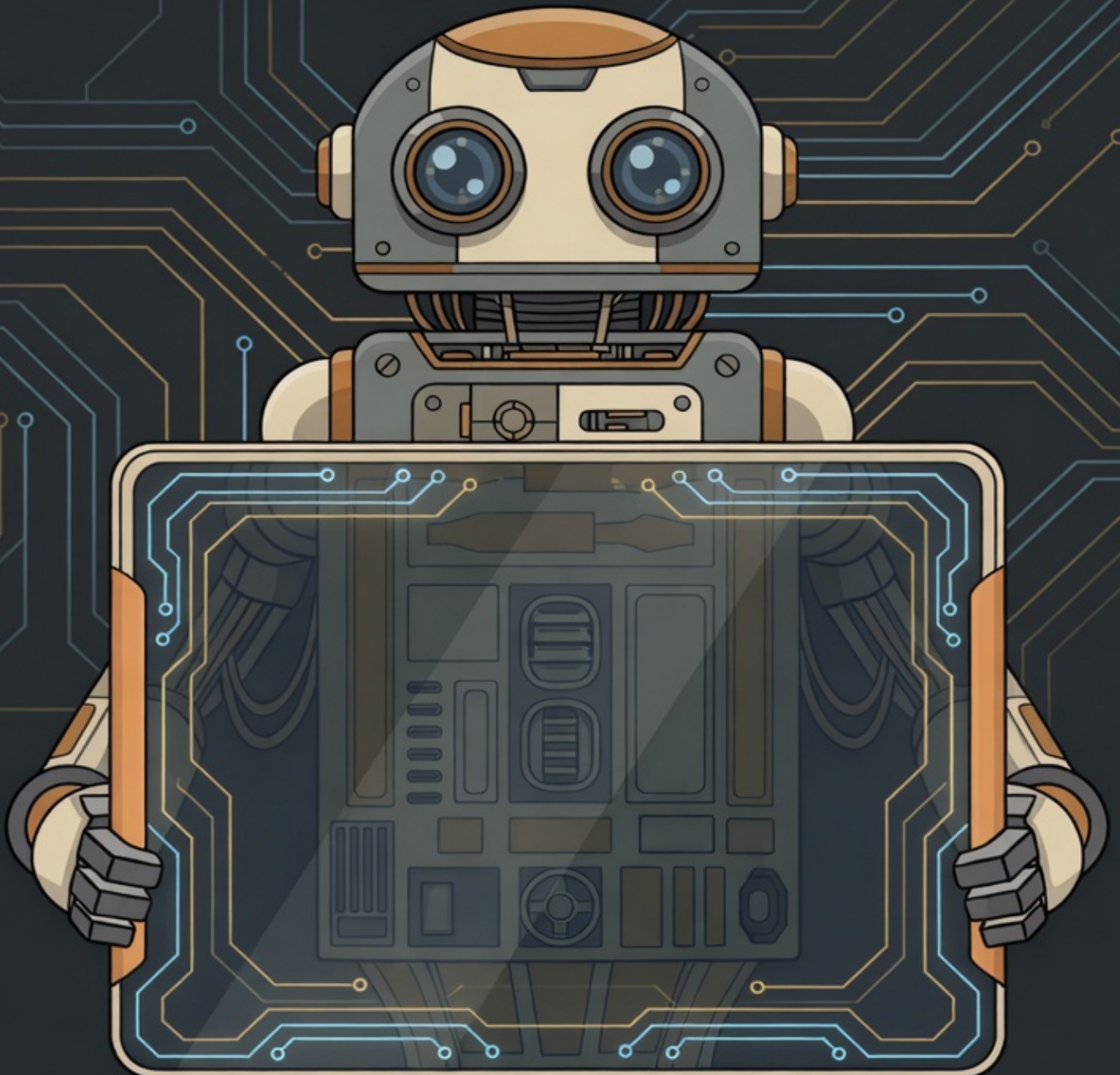


ARLIZ

Go Beyond Arrays, Memory, and Logic —
A Journey from Foundations to Frontiers

Mahdi (m-mdy-m)



ARLIZ

ARRAYS • REASONING • LOGIC • IDENTITY • ZERO

*"From ancient counting stones to quantum algorithms—
every data structure tells the story of human ingenuity."*

LIVING FIRST EDITION

Updated: June 15, 2026

© 2026 Arliz

CREATIVE COMMONS • OPEN SOURCE

ARLIZ

ARRAYS, REASONING, LOGIC, IDENTITY, ZERO

A Living Architecture of Computing — From Voltage to Vector

Author: Mahdi Mamashli (@m-mdy-m), Iran

Repository: <https://github.com/papyrxis/arliz>

Copyright © 2023–2026 **Mahdi Mamashli**. All rights to the prose, explanations, diagrams, and original organization of this work are reserved by the author except as granted under the licenses below.

Dual License. The **book content** (chapters, explanations, diagrams, prose) is licensed under **CC BY-SA 4.0**: you may share and adapt it for any purpose, even commercially, provided you give appropriate credit, link to the license, indicate changes, and distribute derivatives under the same license (<https://creativecommons.org/licenses/by-sa/4.0/>). The **source code, build scripts, and LaTeX tooling** are licensed separately under the **MIT License**, as included in the repository’s LICENSE file.

Preferred Citation. Mahdi Mamashli. (2026). *Arliz: Arrays, Reasoning, Logic, Identity, Zero*. <https://github.com/papyrxis/arliz>

Trademarks. Product, processor, language, and hardware names used throughout this book are the property of their respective owners and are used here only for identification and explanation, with no implied endorsement or affiliation.

No Warranty. This work is provided “AS IS”, without warranty of any kind, express or implied. While every reasonable effort has been made toward technical accuracy, neither the author nor any contributor accepts liability for errors, omissions, or consequences arising from its use. This work is intended for educational and reference purposes.

Contact. Questions about licensing, attribution, or redistribution can be raised via an issue on the repository, or by emailing bitsgenix@gmail.com.

Acknowledgments

To everyone who took the time to read, question, correct,
or argue with an earlier draft of this book
thank you for making the journey sharper and more honest.

Hanieh Bakhshi – hanieh200482@gmail.com

If you contributed to Arliz through code, review, discussion, translation, or simply by catching a mistake and your name does not yet appear here, please open an issue or pull request.

Contents

Title Page	i
Acknowledgments	ii
Contents	iii
About the Author	vi
Preface	vii
Introduction	x
I Substrate & Signal	1
1 The Nature of Data	3
2 The Philosophy of Representation	4
3 Voltage to Bits	5
4 Binary Representation and Boolean Algebra	6
5 Number Systems and Bases	7
6 Integer Representation: Unsigned	8
7 Signed Integer Representation	9
8 Integer Overflow and Wraparound Behavior	10
9 Fixed-Point Representation	11
10 Floating-Point Representation: IEEE 754	12
11 Special Floating-Point Values	13
12 Floating-Point Arithmetic Operations	14
13 Floating-Point Error Analysis	15
14 Decimal Floating-Point Representation	16
15 Extended Precision and Arbitrary Precision	17
16 Character Encoding: ASCII and Extensions	18
17 Unicode: Universal Character Encoding	19
18 Unicode Transformation Formats	20
19 Text Processing Complexities	21

20	Endianness: Byte Ordering	22
21	Cross-Platform Data Exchange	23
22	Bitwise Operations Fundamentals	24
23	Bit Shifting and Rotation	25
24	Bit Manipulation Techniques	26
25	Bit Packing and Flags	27
26	Advanced Bit Hacking	28
27	Data Alignment Fundamentals	29
28	Structure Padding and Layout	30
29	Alignment Optimization Techniques	31
30	Color Representation Models	32
31	Pixel Formats and Bit Depth	33
32	Image Compression Fundamentals	34
33	Video Encoding Principles	35
34	Audio Representation: Sampling Theory	36
35	Audio Encoding Formats	37
36	Signal Processing Representations	38
37	Pointer Representation	39
38	Pointer Arithmetic and Address Calculation	40
39	Special Pointer Values	41
40	Error Detection Codes	42
41	Error Correction Codes	43
42	Cyclic Redundancy Checks (CRC)	44
43	Data Serialization Fundamentals	45
44	Binary Serialization Formats	46
45	Text Serialization Formats	47
46	Custom Binary Protocols	48
47	Data Compression Theory	49

48	Dictionary-Based Compression	50
49	Entropy Coding	51
50	Modern Compression Algorithms	52
51	Specialized Compression	53
	Bibliography	53

About the Author



Mahdi is a backend engineer, open-source developer, and lifelong computer enthusiast from Iran, currently pursuing a Bachelor's degree in Computer Engineering. His fascination with computers began long before university. From the age of four, computers were a constant companion first as a source of curiosity and games, and later as a window into understanding how technology works. Long before writing software, he enjoyed writing stories. At the age of thirteen or fourteen, he wrote his first book, a fictional story centered around dinosaurs. Although far removed from programming, that early experience cultivated a lasting appreciation for exploration, creativity, and the habit of pursuing ideas simply because they were interesting. At nineteen, upon entering

university, a childhood fascination with computers evolved into a serious interest in software engineering. Alongside his academic studies, he began a largely self-directed journey into programming, driven not by formal curricula but by curiosity. Seeking structure for his studies, he created a personal roadmap repository, *Algorithms & Data Structures*, which gradually became both a learning journal and a guide through the broader landscape of computer science.

That journey introduced him to a wide range of programming languages and paradigms. Beyond modern languages, he explored older and less common systems such as Perl, Haskell, Ada, and even a small amount of assembly language. Through C and C++, he built numerous small projects and developed a growing interest in the lower layers of computing how abstractions emerge from hardware and how software ultimately interacts with the machine beneath.

Over the years, his work naturally gravitated toward developer tooling and infrastructure. He has built command-line utilities, backend frameworks, validation systems, HTTP clients, file-tree explorers, moderation services, and other software intended to make development environments simpler and easier to reason about. Much of this work has been released as open source and shared with the broader programming community.

Eventually, a deceptively simple question emerged:

What is an array?

At first, the question appeared trivial. But pursuing it led far beyond the boundaries of introductory programming. Following that thread revealed a path stretching from high-level languages to memory layouts, from CPU caches and transistor physics to GPUs, SIMD instructions, tensor cores, and even the foundations of quantum computation.

Arliz is the written record of that journey an attempt to follow one simple question as far as possible, and to understand not merely how arrays are used, but what they truly are.

Preface

Why Would Anyone Learn This Stuff?

EVERY BOOK HAS ITS ORIGIN STORY, and this one is no exception. If I were to capture the essence of creating this book in a single word, that word would be **curiosity**—though *improvised* comes as a close second. What you hold in your hands (or view on your screen) is the result of years of persistent questioning, a journey that began with a simple yet profound realization: I didn't truly understand what an array was.

This might sound trivial to some. After all, arrays are fundamental to programming, covered in every computer science curriculum, explained in countless tutorials. Yet despite encountering terms like array, stack, queue, hash table, and tensor repeatedly throughout my studies, I found myself increasingly frustrated by the superficial explanations typically offered. Most resources assumed you already knew what these structures fundamentally represented—their conceptual essence, their implementation mechanics, their performance characteristics, and, beneath all of that, the physical machinery that makes any of it possible.

But I wanted the *roots*. I needed to understand not just how to use an array, but what it truly meant at every level—from a voltage difference across a transistor to a tensor core multiplying matrices for a neural network. This led me to a decisive moment:

If I truly want to understand, I must build from the foundation—and follow that foundation all the way to the frontier.

And so began the journey that became Arliz.

The Name and Its Meaning

The name "Arliz" started as a somewhat arbitrary choice—I needed a title, and it sounded right. However, as the book evolved, I discovered a fitting expansion that captures its essence:

Arliz = Arrays, Reasoning, Logic, Identity, Zero

This backronym embodies the core pillars of our exploration:

- **Arrays:** The fundamental data structure we seek to understand from voltage to vector
- **Reasoning:** The systematic thinking behind data organization and algorithmic design
- **Logic:** The formal and physical principles that govern computation and data manipulation
- **Identity:** The concept of distinguishing, indexing, and assigning meaning to elements within structures
- **Zero:** The foundation from which all indexing, computation, and systematic organization originates

You may pronounce it "Ar-liz," "Array-Liz," or however feels natural to you. I personally say "ar-liz," but the pronunciation matters less than the journey it represents.

One Work, Three Volumes

Arliz was never meant to be read in a single sitting, and after enough rewrites I stopped pretending otherwise. What you are holding is not "a book"—it is one volume of a single continuous work, divided the way older technical and philosophical traditions used to divide large projects: not into arbitrary chapters of similar length, but into volumes that each stand on their own while still being unmistakably part of one larger whole. A reader of Ibn Sīnā's *al-Shifā'*, or of Aristotle's or al-Fārābī's treatises, did not expect to carry the entire corpus bound into a single spine; each volume was a complete stage of the argument, readable on its own terms, yet pointing forward and backward to the rest.

Arliz follows that model. The whole work walks one continuous path—from the moment a voltage appears across a transistor, to the moment that voltage becomes a bit, then a number, then an array element, and finally a structure powerful enough to run a sorting algorithm, traverse a graph, or feed a tensor into a neural network. That path is too long, and the material at each stage too dense, to fit comfortably between two covers. So the work is divided into three volumes, each one a complete stage of that path:

- **Volume I — Substrate & Signal:** How information is encoded at all, from semiconductor switching and voltage levels through bits, numbers, characters, and the raw encodings everything else is built from.

- **Volume II — Architecture & Circuitry:** The hardware that turns encoded information into computation—logic gates, memory hierarchies, processors, caches, vector units, GPUs, and the interconnects that move data between all of them.
- **Volume III — Array Odyssey:** Arrays themselves, in full: their mathematics, their memory layout, every variant and technique built on top of them, the data structures and algorithms they enable, and the parallel and large-scale systems that process them today.

This volume’s Introduction lays out exactly what *this* volume covers, what it assumes you already know, and how it connects to its companions; I won’t repeat that here. What matters for this preface is simply that the division into volumes is not a retreat from the project’s scope—if anything, it is what makes the scope possible. Each volume can be read, revised, and improved on its own schedule, while the underlying argument remains one continuous line from voltage to vector.

What This Work Represents

Arliz is not a gentle introduction to programming, nor is it a purely theoretical treatment of data structures. It is a comprehensive technical exploration of the most fundamental data structure in computing, examined from every relevant angle, and grounded the entire way down in the physical reality of the machines that run our code.

This is a living work. As I discover better explanations, uncover new implementation details, or recognize deeper connections between concepts, each volume improves—sometimes independently of the others. Your engagement, through corrections, suggestions, and questions, makes you part of that evolution. The Introduction that follows is the practical companion to this more personal note: it covers prerequisites, difficulty, and how this volume fits into the larger work.

Mahdi Mamashli
2026

“The purpose of abstraction is not to be vague, but to create a new semantic level in which one can be absolutely precise.”

— EDSGER W. DIJKSTRA

Introduction

ARRAYS ARE EVERYWHERE. Every image you view, every text you read, every game you play, every database query you execute, every model that recognizes your voice—arrays underlie them all. Yet despite their ubiquity, arrays remain poorly understood by most programmers. They are treated as primitive constructs, learned hastily and used mechanically, their true nature obscured by layers of abstraction stacked between a line of code and the silicon that executes it.

Arliz exists to remedy that situation. The Preface explained why I went looking for that remedy, what the name means, and why the work is divided into three volumes; this Introduction is about *this* volume specifically—what it covers, what it assumes of you, and how it connects to the rest of the work.

This Volume in the Larger Work

Arliz follows one continuous path: from how information is encoded, to the machines that compute on that information, to the array itself in all its forms. That path is divided into three volumes:

- **Volume I — Substrate & Signal** (this volume)

We begin with the fundamental question: how is information encoded in digital systems at all? We start at the level of voltage and binary switching, then move through binary representation and Boolean algebra, number systems, integer and floating-point formats, character encoding, bitwise operations, alignment, and serialization. This volume establishes the representational substrate that determines how every array element is actually stored and manipulated at the machine level.

- **Volume II — Architecture & Circuitry**

Picks up where this volume ends. Arrays do not exist in abstract space—they live in physical hardware with specific characteristics and constraints. Volume II examines semiconductor physics, logic gates, arithmetic circuits, memory cell design, the full memory hierarchy and cache behavior, processor pipelines,

instruction set architectures, vector and SIMD units, GPU architecture, NUMA systems, and the interconnects that tie it all together.

- **Volume III — Array Odyssey**

The destination of the first two volumes. Arrays in full: their mathematics, memory layout, every major variant, the data structures and algorithms built on top of them, and the parallel and large-scale systems—from CPU threads to GPU kernels to distributed tensors—that process them today.

Each volume is written to be readable on its own, but the dependencies run in one direction: Volume II assumes the representational vocabulary built here in Volume I, and Volume III assumes both. If you find yourself in Volume III wondering why an operation costs what it does at the hardware level, or in Volume II wondering why a value is encoded the way it is, the earlier volume is where that question is answered.

What This Volume Covers

This volume answers a single question, asked as literally as possible: how is information encoded in digital systems, starting from nothing but a voltage difference? We begin before computing altogether—with what data *is*, and what it means for one thing to represent another—and then follow that question through binary switching, Boolean algebra, positional number systems, signed and unsigned integers, fixed- and floating-point arithmetic, character encoding and Unicode, endianness, bitwise manipulation, memory alignment, media representations (color, image, audio, video), pointers, error detection and correction, and serialization and compression. By the end of this volume, you will have every representational tool needed to understand, in Volume II, how hardware actually stores and moves these encodings, and, in Volume III, how arrays organize them into the structures that power real software.

Prerequisites

This book assumes substantial preparation:

Mathematical Maturity: You should be comfortable with discrete mathematics, linear algebra, and basic mathematical analysis. Specifically, you should have completed (or be comfortable with) the material in *Mathesis: The Mathematical Foundations of Computing*.

Algorithmic Analysis: You should understand asymptotic notation, recurrence relations, and basic complexity analysis. The material in *The Art of Algorithmic Analysis* provides the necessary foundations.

Programming Experience: You should be proficient in at least one programming language and comfortable reading code in multiple languages. We use pseudocode, C, and occasionally other languages for examples.

Without these prerequisites, you will struggle with substantial portions of this volume. With them, you are prepared for a deep, rewarding start to the journey from voltage to vector.

The Living Nature of This Work

Check the GitHub repository regularly for updates. The version you read today will be superseded by improved versions tomorrow—and because the work is now split across volumes, each one can be revised on its own schedule without forcing a reprint of the others. This is not a weakness but a strength: the work remains current, accurate, and relevant, and your corrections and questions are part of how it gets there.

A Note on Difficulty

This volume is challenging by design, even though it is, in a sense, the most "elementary" of the three—it deals with representation rather than algorithms or hardware. But representation is where every later mistake is born: a misunderstood encoding produces bugs and performance problems that no amount of algorithmic cleverness can fix later. Some sections will require sustained effort to master.

This difficulty is intentional. Genuine understanding is never cheap—it demands intellectual investment, persistent effort, and willingness to struggle with complex concepts. If you find sections difficult, that is normal. Persist. Work through examples. Understanding will come through sustained engagement, and it will pay off directly when you reach Volumes II and III.

Begin

This volume is the first stage of a longer journey—the one that starts at a voltage and, three volumes later, ends at a tensor. Everything here is foundation: necessary, occasionally dense, and built to be returned to.

Welcome to **Arliz**. Let us begin where everything begins—at the bit.

"Premature optimization is the root of all evil."

— DONALD E. KNUTH

Part I

Substrate & Signal

***B**EFORE WE can understand how arrays store elements, we must first understand how computers represent information itself. Every piece of data—numbers, text, images, instructions—exists as patterns of bits, and every bit begins its life as a voltage. This volume follows that single thread: from a voltage difference across a transistor, through binary encoding, to the numbers, characters, and bit-level structures that everything else in this work is built from.*

“The choice of representation is often more important than the choice of algorithm.”

— DONALD E. KNUTH

Chapter 1

The Nature of Data

What data actually is before it is encoded: the distinction between data, information, and meaning; raw measurements versus interpreted values; data as the record of a distinction or a difference; the role of an observer or interpreter in turning a physical trace into data; examples spanning natural "data" (tree rings, sediment layers, starlight) and human-made data (tally marks, ledgers, sensor readings); why data alone is inert without a scheme to read it; the seed of the idea that everything downstream—bits, numbers, arrays—is data wrapped in successive layers of agreed-upon meaning.

Chapter 2

The Philosophy of Representation

What it means for one thing to stand for another; concrete versus abstract representation and the historical shift from tally marks to abstract number systems; Plato's allegory of the cave and the idea that representations can be incomplete or misleading; Aristotle's more practical view of mental representation as a useful internal model rather than a perfect copy; medieval and modern philosophical treatments of representation (Aquinas, Descartes, Kant, Sartre) and their relevance to encoding and interpretation; the layered nature of representation, where symbols represent symbols all the way down to physical reality; the abstraction hierarchy in computing, from physical voltages to applications, and where arrays sit within it; Shannon's information theory as the mathematical formalization of representation, encoding, decoding, and the bit as the fundamental unit of distinguishing information.

Chapter 3

Voltage to Bits

How transistors encode binary states, voltage levels, noise margins, signal integrity, why binary won over ternary.

Chapter 4

Binary Representation and Boolean Algebra

Two-state logic, Boolean operations, truth tables, De Morgan's laws, bit as fundamental unit.

Chapter 5

Number Systems and Bases

Positional notation, decimal, binary, octal, hexadecimal. Base conversion algorithms. Historical development. Why different bases matter for different purposes.

Chapter 6

Integer Representation: Unsigned

Binary positional notation, range limitations, conversion algorithms, hexadecimal convenience.

Chapter 7

Signed Integer Representation

Sign-magnitude, one's complement, two's complement (why it won), arithmetic operations, overflow detection.

Chapter 8

Integer Overflow and Wraparound Behavior

Undefined behavior, modular arithmetic, detecting overflow, saturating arithmetic, language-specific behaviors.

Chapter 9

Fixed-Point Representation

Q-format notation, scaling factors, precision-range tradeoffs, embedded systems applications, fixed-point arithmetic.

Chapter 10

Floating-Point Representation: IEEE 754

Sign, exponent, mantissa encoding, normalized and denormalized numbers, single vs. double precision.

Chapter 11

Special Floating-Point Values

Infinity (positive and negative), NaN (quiet and signaling), signed zero, subnormal numbers.

Chapter 12

Floating-Point Arithmetic Operations

Addition, multiplication, division, rounding modes (round to nearest, toward zero, toward), fused multiply-add.

Chapter 13

Floating-Point Error Analysis

Precision loss, catastrophic cancellation, machine epsilon, relative error, Kahan summation algorithm.

Chapter 14

Decimal Floating-Point Representation

IEEE 754-2008 decimal formats, financial computing requirements, densely packed decimal.

Chapter 15

Extended Precision and Arbitrary Precision

80-bit extended precision, quadruple precision (128-bit), arbitrary precision libraries (GMP, MPFR).

Chapter 16

Character Encoding: ASCII and Extensions

7-bit ASCII, extended ASCII variants, code pages, limitations for internationalization.

Chapter 17

Unicode: Universal Character Encoding

Code points, planes, combining characters, grapheme clusters, normalization forms (NFC, NFD, NFKC, NFKD).

Chapter 18

Unicode Transformation Formats

UTF-8 (variable length, backward compatible), UTF-16 (surrogate pairs), UTF-32 (fixed length), BOM issues.

Chapter 19

Text Processing Complexities

Grapheme vs. code point counting, case folding, locale-dependent operations, text segmentation.

Chapter 20

Endianness: Byte Ordering

Big-endian vs. little-endian, historical reasons, network byte order, bi-endian systems, byte swapping.

Chapter 21

Cross-Platform Data Exchange

Serialization formats, portable binary formats, protocol design considerations.

Chapter 22

Bitwise Operations Fundamentals

AND, OR, XOR, NOT operations, truth tables, bit manipulation primitives.

Chapter 23

Bit Shifting and Rotation

Logical shift, arithmetic shift, rotate left/right, applications to multiplication/division by powers of 2.

Chapter 24

Bit Manipulation Techniques

Setting/clearing/toggling bits, bit masks, extracting bit fields, counting set bits (population count).

Chapter 25

Bit Packing and Flags

Packing multiple boolean flags, bitfields in structs, union tricks, bit arrays.

Chapter 26

Advanced Bit Hacking

Finding rightmost set bit, isolating bit patterns, bit reversal, Gray code conversion.

Chapter 27

Data Alignment Fundamentals

Natural alignment, alignment requirements by type, misalignment penalties.

Chapter 28

Structure Padding and Layout

Compiler padding rules, struct member ordering, packing pragmas, cache line awareness.

Chapter 29

Alignment Optimization Techniques

Manual padding, alignment attributes, reordering for cache efficiency.

Chapter 30

Color Representation Models

RGB, RGBA, HSV, HSL, CMYK, YUV, color spaces, gamma correction.

Chapter 31

Pixel Formats and Bit Depth

1-bit, 8-bit indexed, 16-bit (RGB565), 24-bit, 32-bit (with alpha), HDR formats.

Chapter 32

Image Compression Fundamentals

Lossless (PNG, GIF) vs. lossy (JPEG), compression ratios, quality tradeoffs.

Chapter 33

Video Encoding Principles

Frame types (I, P, B), motion compensation, codecs (H.264, H.265, VP9, AV1).

Chapter 34

Audio Representation: Sampling Theory

Nyquist theorem, sample rate, bit depth, quantization noise, aliasing.

Chapter 35

Audio Encoding Formats

PCM (uncompressed), FLAC (lossless), MP3, AAC, Opus (lossy), perceptual coding.

Chapter 36

Signal Processing Representations

Time domain, frequency domain, spectrograms, wavelet transforms.

Chapter 37

Pointer Representation

How pointers are stored in memory, pointer size (32-bit vs. 64-bit), pointer tagging.

Chapter 38

Pointer Arithmetic and Address Calculation

Array element addressing, struct member offsets, pointer subtraction.

Chapter 39

Special Pointer Values

Null pointers, wild pointers, dangling pointers, pointer alignment requirements.

Chapter 40

Error Detection Codes

Parity bits (even/odd), checksums, CRC algorithms, hash-based integrity.

Chapter 41

Error Correction Codes

Hamming codes, Reed-Solomon codes, LDPC codes, convolutional codes.

Chapter 42

Cyclic Redundancy Checks (CRC)

CRC polynomials, CRC-8, CRC-16, CRC-32 standards, efficient implementation using tables, applications in data integrity.

Chapter 43

Data Serialization Fundamentals

Converting in-memory structures to byte streams, portability issues.

Chapter 44

Binary Serialization Formats

Protocol Buffers, FlatBuffers, Cap'n Proto, MessagePack, CBOR.

Chapter 45

Text Serialization Formats

JSON, XML, YAML, TOML, human-readable vs. machine-efficient tradeoffs.

Chapter 46

Custom Binary Protocols

Designing efficient binary formats, alignment considerations, versioning.

Chapter 47

Data Compression Theory

Information theory basics, entropy, lossless vs. lossy bounds.

Chapter 48

Dictionary-Based Compression

LZ77, LZ78, LZW, Lempel-Ziv family, dictionary construction.

Chapter 49

Entropy Coding

Huffman coding, arithmetic coding, asymmetric numeral systems (ANS).

Chapter 50

Modern Compression Algorithms

zlib, gzip, bzip2, LZMA, Zstandard, Brotli, compression levels.

Chapter 51

Specialized Compression

Run-length encoding, delta encoding, columnar compression, domain-specific methods.